ANKARA ÜNİVERSİTESİ

MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BLM4522

AĞ TABANLI PARALEL DAĞITIM SİSTEMLERİ

PROJE 1

VERİTABANI PERFORMANS OPTİMİZASYONU VE İZLEME

Zeliha Kıyak

22290878

GitHub **:https://github.com/zelihakiyak**

VİDEO LİNKİ: https://youtu.be/UoykTWt9ddc

1. Giriş

Bu projede AdventureWorks2022 veritabanı üzerinde performans analizi yapılmıştır. Proje kapsamında Dynamic Management Views (DMV) araçlarıyla yük testi ve sorgu izleme gerçekleştirilmiştir. Execution Plan ile darboğazlar tespit edilmiş olup indeks optimizasyonu uygulanmıştır. Gereksiz indeksler kaldırılmış ve kullanıcı erişim rolleri oluşturulmuştur.

2. Veritabanı İzleme DMV Analizi

‘sys.dm_exec_query_stats’ ve ‘sys.dm_exec_sql_text’ DMV'leri kullanılarak SQL Server'ın o ana kadar çalıştırdığı sorgular analiz edildi. Ortalama çalışma süresi (avg_elapsed_time) ve ortalama mantıksal okuma (avg_logical_reads) değerleri en yüksek olan ilk 10 sorgu belirlendi.

```
SELECT TOP 10
    qs.total_elapsed_time / qs.execution_count AS avg_elapsed_time,
    qs.execution_count,
    qs.total_logical_reads / qs.execution_count AS avg_logical_reads,
    SUBSTRING(st.text, ...) AS query_text
FROM sys.dm_exec_query_stats AS qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) AS st
ORDER BY avg_elapsed_time DESC;
```

Veritabanına yeni bağlanıldığından DMV istatistikleri henüz zengin değildi. Sistemi zorlamak için kasıtlı olarak ağır 4 sorgu çalıştırıldı.

```
-- Sorgu 1: GROUP BY + JOIN
SELECT
    p.FirstName + ' ' + p.LastName AS FullName,
    COUNT(soh.SalesOrderID) AS OrderCount,
    SUM(soh.TotalDue) AS TotalSpent
FROM Person.Person p
JOIN Sales.Customer c ON p.BusinessEntityID = c.PersonID
JOIN Sales.SalesOrderHeader soh ON c.CustomerID = soh.CustomerID
GROUP BY p.FirstName, p.LastName
ORDER BY TotalSpent DESC;

-- Sorgu 2: LIKE ile tablo tarama
SELECT * FROM Person.Person
WHERE FirstName LIKE '%an%'
AND LastName LIKE '%on%';
```

```
-- Sorgu 3: Fonksiyon içinde kolon
SELECT * FROM Sales.SalesOrderHeader
WHERE YEAR(OrderDate) = 2011
AND MONTH(OrderDate) = 6;

-- Sorgu 4: Büyük JOIN zinciri
SELECT
    sod.ProductID,
    p.Name AS ProductName,
    SUM(sod.OrderQty) AS TotalQty,
    SUM(sod.LineTotal) AS TotalRevenue
FROM Sales.SalesOrderDetail sod
JOIN Production.Product p ON sod.ProductID = p.ProductID
JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID =
soh.SalesOrderID
JOIN Sales.Customer c ON soh.CustomerID = c.CustomerID
JOIN Person.Person per ON c.PersonID = per.BusinessEntityID
GROUP BY sod.ProductID, p.Name
ORDER BY TotalRevenue DESC;
```

Yavaş sorgu tespiti için oluşturulan sorguyu yeniden yürüttüğümüzde sonuçların değiştiği görüldü.

Sıralama	avg_elapsed_time	avg_logical_reads	Yorum
1	200.939 µs	1.491 sayfa	En yavaş (GROUP BY + JOIN)
2	161.446 µs	3.615 sayfa	En yüksek okuma (LIKE ile tarama)
3	99.103 µs	686 sayfa	Orta düzey

3.615 sayfa okuma kritik bir sorun işaretidir. SQL Server veriyi bulmak için 3.615 veriyi tek tek taradı. İndeks olsaydı bu sayı 10-20 sayfaya düşerdi. Bu yüzden LIKE '%an%' içeren sorgu Ctrl+M ile çalıştırılarak gerçek yürütme planı incelendi. Aşağıdaki operatörler tespit edildi:

Operatör	Anlamı	Sorun
Index Scan (Nonclustered)	İndeks baştan sona taranmış	LIKE'ın baştaki % operatörü SQL Server'ın nereden başlayacağını bilmesini engeller
Key Lookup (Clustered)	İndekste bulunan her satır için ana tabloya geri dönülmüş	Çift okuma maliyeti
Nested Loops	Her satır için döngü kurulmuş	Büyük tablolarda ciddi performans kaybı

Index oluşturmak için sorguyu yürüttüğümüzde en yüksek logical reads değeri 3.593 olan sorgu, LIKE operatörü içerdiğinden indekslenemedi. Bu nedenle indekslenebildiğini doğruladığımız 3. sıradaki SalesOrderHeader sorgusuna odaklanıldı.

```
-- Eksik İndeks Tespiti
SELECT TOP 10
    mid.statement AS table_name,
    migs.avg_total_user_cost * migs.avg_user_impact *
        (migs.user_seeks + migs.user_scans) AS improvement_measure,
    'CREATE INDEX IX_missing_' +
        CAST(ROW_NUMBER() OVER(ORDER BY migs.avg_user_impact DESC) AS
VARCHAR) +
        ' ON ' + mid.statement +
        ' (' + ISNULL(mid.equality_columns, '') +
        CASE WHEN mid.inequality_columns IS NOT NULL
             THEN (CASE WHEN mid.equality_columns IS NOT NULL
                        THEN ',' ELSE '' END) +
mid.inequality_columns
             ELSE '' END + ')' +
        ISNULL(' INCLUDE (' + mid.included_columns + ')', '') AS
create_index_statement,
    migs.avg_user_impact AS expected_improvement_percent
FROM sys.dm_db_missing_index_details mid
JOIN sys.dm_db_missing_index_groups mig ON mid.index_handle =
mig.index_handle
JOIN sys.dm_db_missing_index_group_stats migs ON
mig.index_group_handle = migs.group_handle
WHERE mid.database_id = DB_ID()
ORDER BY improvement_measure DESC;
```

3. Sorgu İyileştirme

sys.dm_db_missing_index_details DMV'si ile SQL Server'ın önerdiği indeksler sorgulandı. Beklenen iyileşme oranı %77,96 olarak görüldü. İndeksi oluşturmadan önce sorgunun mevcut hızı SET STATISTICS IO ile ölçüldü. İndeks oluşturduktan sonra tekrar ölçüldüğünde karşımıza çıkan tablo şu şekilde:

Metrik	Indekssiz	Indeksli
Logical Reads	686 sayfa	9 sayfa
Elapsed Time	117 ms	218 ms
CPU Time	16 ms	0 ms

```
-- Indekssiz olcum
SET STATISTICS TIME ON;
SET STATISTICS IO ON;
SELECT soh.CustomerID, soh.OrderDate, soh.TotalDue, soh.Status
FROM Sales.SalesOrderHeader soh
```

```
WHERE soh.OrderDate >= '2011-01-01'
AND soh.OrderDate < '2012-01-01'
AND soh.Status = 5
ORDER BY soh.TotalDue DESC;
SET STATISTICS TIME OFF;
SET STATISTICS IO OFF;

-- Indeksi olusturma
CREATE INDEX IX_missing_1
ON Sales.SalesOrderHeader ([Status], [OrderDate])
INCLUDE ([CustomerID], [TotalDue]);
```

Elapsed time ilk çalıştırmada cold cache etkisi nedeniyle artmış görünse de CPU süresinin sıfıra inmesi ve logical reads'in %98.7 azalması indeksin etkin oluşunu kanıtlamaktadır. Yine de sistemde her zaman indeks eklemek iyi bir çözüm değildir. Gereksiz indeksler yazma maliyetini artırır.

4. İndeks Yönetimi

sys.dm_db_index_usage_stats DMV'si ile hiç kullanılmamış indeksler tespit edildi. user_seeks + user_scans + user_lookups = 0 olan indeksler listelendi.

```
-- Gereksiz indeks tespiti
SELECT TOP 10
    OBJECT_NAME(i.object_id) AS table_name,
    i.name AS index_name,
    i.type_desc AS index_type,
    ius.user_seeks,
    ius.user_scans,
    ius.user_lookups,
    ius.user_updates
FROM sys.indexes i
LEFT JOIN sys.dm_db_index_usage_stats ius
    ON i.object_id = ius.object_id
    AND i.index_id = ius.index_id
    AND ius.database_id = DB_ID()
WHERE OBJECT_NAME(i.object_id) NOT LIKE 'sys%'
AND i.type_desc != 'HEAP'
AND i.is_primary_key = 0
AND i.is_unique = 0
AND ISNULL(ius.user_seeks, 0) = 0
AND ISNULL(ius.user_scans, 0) = 0
```

```
AND ISNULL(ius.user_lookups, 0) = 0
ORDER BY ISNULL(ius.user_updates, 0) DESC;
```

61 adet kullanılmamış indeks tespit edildi. Kullanılmayan indeksler INSERT, UPDATE ve DELETE işlemlerini yavaşlattığı, disk alanı tükettiği ve sorgu planlayıcısını yanılttığı için kaldırılması önerilir. Kullanılmayan indeksler listelenirken sistem tabloları (ledger, filestream) elenmiş, yalnızca uygulama tablolarına ait indeksler değerlendirmeye alınmıştır. IX_Address_StateProvinceID indeksinin normal bir uygulama tablosuna ait olması, hiç kullanılmamış olması ve güvenle kaldırılabilir yapıda olması nedeniyle seçilmiştir.

```
-- İndeks Tanımını Görme
SELECT
    i.name AS index_name,
    i.type_desc,
    COL_NAME(ic.object_id, ic.column_id) AS column_name,
    ic.is_included_column
FROM sys.indexes i
JOIN sys.index_columns ic ON i.object_id = ic.object_id
    AND i.index_id = ic.index_id
WHERE i.name = 'IX_Address_StateProvinceID'
AND OBJECT_NAME(i.object_id) = 'Address';
```

İndeks kaldırılmadan önce tanımı incelenerek hangi kolonlar üzerinde kurulu olduğu not alındı.

```
-- Indeksi kaldir
DROP INDEX IX_Address_StateProvinceID ON Person.Address;


-- Dogrulama: 0 satir gelmeli
SELECT name FROM sys.indexes
WHERE OBJECT_NAME(object_id) = 'Address'
AND name = 'IX_Address_StateProvinceID';
```

Ardından DROP INDEX komutuyla kaldırıldı. Son olarak doğrulama sorgusu çalıştırıldı ve 0 satır dönerek indeksin başarıyla silindiği teyit edildi.

5. Veri Yöneticisi Rolleri

Veritabanı güvenliği için farklı yetki seviyelerine sahip kullanıcı rolleri oluşturulmuştur. Bu projede 3 farklı yetki seviyesinde kullanıcı rolü oluşturuldu:

Kullanıcı	Rol	Yetkiler	Erişebileceği Şemalar
analyst_user	Analist	SELECT	Sales, Person
app_user	Uygulama Kullanıcısı	SELECT, INSERT, UPDATE	Sales
db_admin	Veritabanı Yöneticisi	db_owner (Tam yetki)	Tüm şemalar

```
USE master;
CREATE LOGIN analyst_user WITH PASSWORD = 'Analyst123!';
CREATE LOGIN app_user WITH PASSWORD = 'AppUser123!';
CREATE LOGIN db_admin WITH PASSWORD = 'DbAdmin123!';


USE AdventureWorks2022;
CREATE USER analyst_user FOR LOGIN analyst_user;
CREATE USER app_user FOR LOGIN app_user;
CREATE USER db_admin FOR LOGIN db_admin;


-- Yetkileri atama
GRANT SELECT ON SCHEMA::Sales TO analyst_user;
GRANT SELECT ON SCHEMA::Person TO analyst_user;
GRANT SELECT, INSERT, UPDATE ON SCHEMA::Sales TO app_user;
EXEC sp_addrolemember 'db_owner', 'db_admin';
```

Önce sunucu seviyesinde login, ardından veritabanı seviyesinde user oluşturuldu. Login SQL Server'a bağlanma iznini, user ise ilgili veritabanına erişimi tanımlar. Her kullanıcıya minimum yetki prensibi doğrultusunda farklı yetkiler atandı. Doğrulama sorgusu 8 satır döndürerek tüm yetkilerin başarıyla tanımlandığını teyit etti.